



UNIVERSIDAD DE LOS ANDES
Facultad de Ingeniería y Ciencias Aplicadas
Arquitectura de Computadores 2026-20

Proyecto 1 — Diseño Lógico y FPGA

Calculadora de 4 bits

Grupo N° 1

Integrantes:

Cristian Acevedo

Guillermo Arriaza

Eduardo Ponce

Profesor:

Jorge Gomez

8 de Septiembre de 2026

Resumen

El presente informe describe el diseño e implementación de una calculadora digital de 4 bits desarrollada como parte del Proyecto 1 del curso Arquitectura de Computadores.

El sistema fue implementado en lenguaje Verilog para una FPGA Lattice iCE40 HX1K de la Nandland Go Board. La calculadora permite ejecutar operaciones de reinicio, suma, resta, resta inversa y desplazamientos lógicos, trabajando con datos de cuatro bits e interpretación de valores negativos mediante complemento a dos.

Una de las principales restricciones del proyecto consiste en implementar la lógica combinacional utilizando compuertas lógicas básicas, evitando operadores aritméticos y de desplazamiento de alto nivel en los módulos sintetizables. En consecuencia, las operaciones se construyen mediante primitivas lógicas como AND, OR, NOT, XOR y BUF.

El diseño se organiza en una unidad de procesamiento, una unidad de control y una interfaz física. Dentro de la unidad de procesamiento se encuentran el sumador de cuatro bits, los restadores, los desplazadores, el multiplexor de selección del segundo operando, la ALU y el registro encargado de almacenar el resultado.

En este informe se presentan la arquitectura general del sistema, las tablas de verdad, expresiones booleanas y mapas de Karnaugh utilizados en el diseño, además de la implementación lógica de las operaciones principales de la calculadora.

Índice

1. Objetivo del proyecto	4
2. Operaciones implementadas	4
3. Arquitectura general de la calculadora	6
3.1. Diagrama general	7
3.2. Secuencia de funcionamiento	7
4. Diseño lógico del Full Adder	8
4.1. Tabla de verdad	8
4.2. Expresión booleana de la suma	8
4.3. Expresión booleana del acarreo	9
4.4. Mapas de Karnaugh	10
4.4.1. Mapa de Karnaugh de la salida S	10
4.4.2. Mapa de Karnaugh de la salida C_{out}	11
5. Implementación de las operaciones a nivel de compuertas	11
5.1. Sumador de cuatro bits	12
5.2. Resta mediante complemento a dos	12
5.3. Resta inversa	13
5.4. Multiplexor para selección del segundo operando	14
5.5. Desplazamiento lógico a la izquierda	15
5.6. Desplazamiento lógico a la derecha	17
5.7. Selección de operaciones en la ALU	18
6. Almacenamiento del resultado y realimentación	19



7. Interfaz con la FPGA	20
7.1. Botones de entrada	21
7.2. Tratamiento de los botones	21
7.3. Indicadores LED	22
7.4. Displays de siete segmentos	22
8. Restricciones de implementación	23
9. Conclusiones	23
10. Declaración de uso de Inteligencia Artificial	25
10.1. Identificación de la actividad	25
10.2. Declaración sobre uso de IA	25
10.3. Descripción de uso de IA	25
10.4. Declaración de responsabilidad académica	26

1. Objetivo del proyecto

El objetivo del proyecto es diseñar e implementar una calculadora digital de 4 bits utilizando Verilog, cuya lógica combinacional se construye mediante compuertas lógicas básicas.

El sistema se implementa sobre una FPGA Lattice iCE40 HX1K de la Nandland Go Board.

La calculadora recibe operandos de cuatro bits y permite interpretar valores negativos mediante representación en complemento a dos.

Cada operación genera un resultado de cuatro bits, el cual se almacena en un registro independiente. Este resultado puede utilizarse posteriormente como segundo operando de una nueva operación, permitiendo encadenar cálculos sucesivos.

Además, el diseño busca aplicar conceptos fundamentales de sistemas digitales, tales como:

- álgebra booleana;
- tablas de verdad;
- mapas de Karnaugh;
- lógica combinacional;
- multiplexores;
- sumadores binarios;
- registros;
- máquinas de estados;
- implementación de sistemas digitales en FPGA.

2. Operaciones implementadas

La calculadora implementa seis códigos de operación.

Cuadro 1: Operaciones implementadas

Código	Operación	Resultado
000	Reinicio	$R = 0000$
001	Suma	$R = A + B$
010	Resta	$R = A - B$
011	Resta inversa	$R = B - A$
100	Shift left lógico	$R = A \ll B[1 : 0]$
101	Shift right lógico	$R = A \gg B[1 : 0]$

Fuente: Elaboración propia.

Los desplazamientos utilizan únicamente los dos bits menos significativos del segundo operando, $B[1 : 0]$, permitiendo desplazar el valor A entre cero y tres posiciones.

Todos los cálculos se realizan con un ancho fijo de cuatro bits. Si una operación produce un resultado de mayor tamaño, únicamente se conservan los cuatro bits menos significativos.

Por ejemplo:

$$7 + 3 = 10$$

En cuatro bits:

$$0111 + 0011 = 1010$$

El patrón binario 1010, interpretado en complemento a dos de cuatro bits, corresponde al valor -6 .

Por lo tanto, el diseño mantiene el comportamiento natural de overflow asociado al ancho fijo de cuatro bits.

3. Arquitectura general de la calculadora

El sistema se organiza en diferentes bloques funcionales que permiten seleccionar una operación, ingresar los operandos, ejecutar el cálculo y almacenar el resultado.

La arquitectura puede dividirse en tres partes principales:

- **Unidad de control:** gestiona la secuencia de ingreso mediante una máquina de estados.
- **Unidad de procesamiento:** contiene el multiplexor de operandos, la ALU y el registro de resultado.
- **Interfaz de entrada y salida:** conecta los botones, LEDs y displays de siete segmentos de la FPGA con el sistema digital.

El usuario comienza seleccionando el código de operación. Posteriormente ingresa el primer operando A y luego el segundo operando B .

Antes de ingresar a la ALU, el segundo operando pasa por un multiplexor que permite elegir entre:

- el valor B ingresado por el usuario;
- el resultado almacenado de la operación anterior.

La ALU recibe los dos operandos y el código de operación.

Finalmente, el resultado generado por la ALU se almacena en un registro de cuatro bits.

La salida de este registro cumple dos funciones:

1. constituye el resultado actual de la calculadora;
2. puede realimentarse como segundo operando de una operación posterior.

3.1. Diagrama general

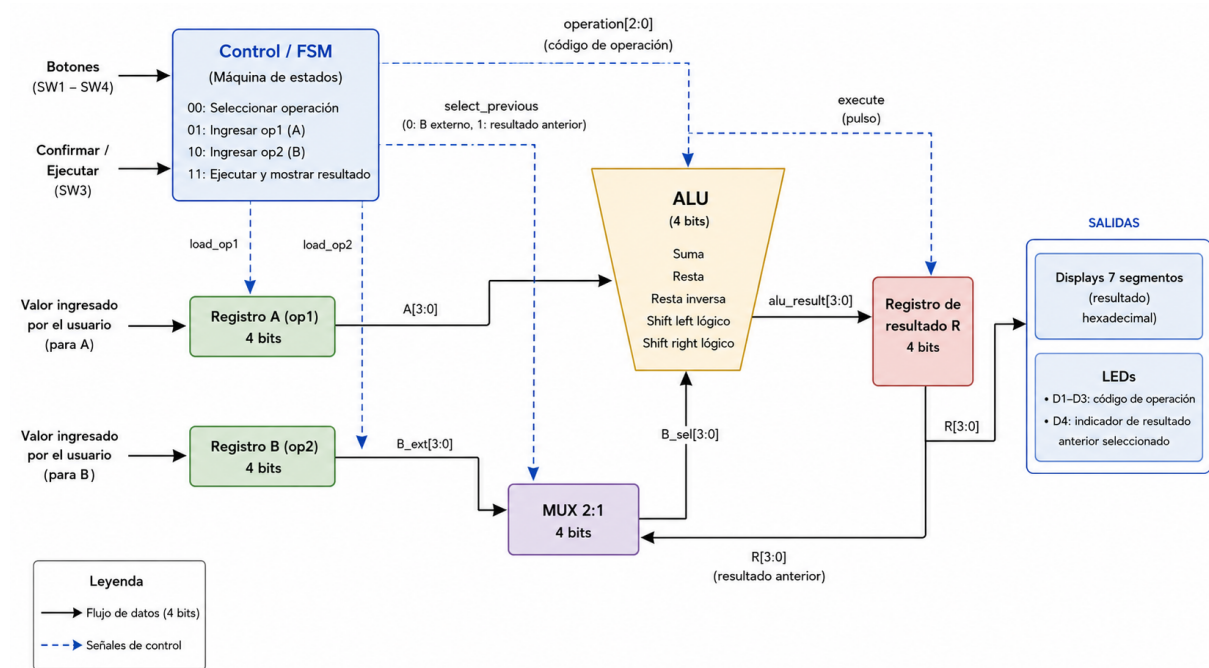


Figura 1: Diagrama general de arquitectura de la calculadora.

3.2. Secuencia de funcionamiento

La interacción con la calculadora se organiza mediante cuatro estados principales.

Cuadro 2: Estados principales del sistema

Estado	Función
00	Selección del código de operación
01	Ingreso y almacenamiento del primer operando
10	Ingreso y almacenamiento del segundo operando
11	Ejecución y visualización del resultado

El botón de confirmación permite avanzar entre estos estados.

Al finalizar una operación, una nueva confirmación devuelve el sistema al estado inicial para comenzar un nuevo cálculo.

4. Diseño lógico del Full Adder

El bloque `full_adder` constituye la unidad fundamental utilizada para implementar las operaciones aritméticas del sistema.

El circuito recibe tres entradas de un bit:

$$A, \quad B, \quad C_{in}$$

y genera dos salidas:

$$S, \quad C_{out}$$

donde S corresponde al bit de suma y C_{out} al acarreo de salida.

4.1. Tabla de verdad

Cuadro 3: Tabla de verdad del Full Adder

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Fuente: Elaboración propia.

4.2. Expresión booleana de la suma

La salida S toma el valor lógico 1 cuando existe una cantidad impar de entradas activas.

Por lo tanto:

$$\boxed{S = A \oplus B \oplus C_{in}}$$

La implementación se realiza mediante dos compuertas XOR consecutivas.

Primero:

$$X = A \oplus B$$

y posteriormente:

$$S = X \oplus C_{in}$$

Por consiguiente:

$$S = (A \oplus B) \oplus C_{in}$$

4.3. Expresión booleana del acarreo

La salida C_{out} se activa cuando al menos dos de las tres entradas se encuentran en nivel lógico alto.

Una expresión equivalente es:

$$\boxed{C_{out} = AB + AC_{in} + BC_{in}}$$

La implementación utilizada puede expresarse como:

$$\boxed{C_{out} = AB + C_{in}(A \oplus B)}$$

Primero:

$$X = A \oplus B$$

Luego:

$$P_1 = AB$$

$$P_2 = C_{in}X$$

Finalmente:

$$C_{out} = P_1 + P_2$$

Esta estructura requiere únicamente compuertas XOR, AND y OR.

4.4. Mapas de Karnaugh

Las columnas de los mapas se ordenan mediante código Gray:

$$BC_{in} = 00, 01, 11, 10$$

4.4.1. Mapa de Karnaugh de la salida S

Los minterms para la salida de suma son:

$$S = \Sigma m(1, 2, 4, 7)$$

Cuadro 4: Mapa de Karnaugh de la salida S

$A \backslash BC_{in}$	00	01	11	10
0	0	1	0	1
1	1	0	1	0

Debido al comportamiento de paridad de esta función, los unos no pueden agruparse en potencias de dos mayores a uno.

Su representación compacta mediante XOR es:

$$S = A \oplus B \oplus C_{in}$$

4.4.2. Mapa de Karnaugh de la salida C_{out}

Los minterms correspondientes al acarreo son:

$$C_{out} = \Sigma m(3, 5, 6, 7)$$

Cuadro 5: Mapa de Karnaugh de la salida C_{out}

$A \backslash BC_{in}$	00	01	11	10
0	0	0	1	0
1	0	1	1	1

Del mapa se obtiene:

$$C_{out} = AB + AC_{in} + BC_{in}$$

5. Implementación de las operaciones a nivel de compuertas

La ALU se construye a partir de bloques combinacionales independientes.

Cada operación genera un resultado de cuatro bits y posteriormente una red de selección determina qué resultado llega a la salida de la ALU.

En los módulos sintetizables no se utilizan directamente los operadores aritméticos $+$ o $-$ ni los operadores de desplazamiento de Verilog.

5.1. Sumador de cuatro bits

El módulo de suma de cuatro bits se construye conectando en cascada cuatro bloques `full_adder`.

Esta arquitectura corresponde a un sumador de acarreo propagado, o *ripple-carry adder*.

Para el primer bit:

$$(A_0, B_0, C_{in}) \rightarrow (S_0, C_1)$$

Para el segundo bit:

$$(A_1, B_1, C_1) \rightarrow (S_1, C_2)$$

Para el tercer bit:

$$(A_2, B_2, C_2) \rightarrow (S_2, C_3)$$

Finalmente:

$$(A_3, B_3, C_3) \rightarrow (S_3, C_{out})$$

El resultado corresponde a:

$$S = [S_3 \ S_2 \ S_1 \ S_0]$$

En una suma normal, el acarreo inicial se fija en:

$$C_{in} = 0$$

El acarreo final C_{out} no se incorpora al registro de resultado, debido a que la calculadora utiliza un ancho fijo de cuatro bits.

5.2. Resta mediante complemento a dos

La operación:

$$A - B$$

se implementa sin utilizar directamente el operador de resta.

Para ello se emplea la identidad de complemento a dos:

$$A - B = A + \overline{B} + 1$$

Primero, cada bit del operando B es invertido:

$$\overline{B_i} = \neg B_i$$

Luego, el resultado invertido se conecta al sumador de cuatro bits utilizando:

$$C_{in} = 1$$

Por consiguiente:

$$\boxed{R = A + \overline{B} + 1}$$

La inversión se realiza mediante cuatro compuertas NOT y la suma mediante los cuatro `full_adder` ya descritos.

Esta estructura permite obtener tanto resultados positivos como negativos utilizando representación en complemento a dos.

5.3. Resta inversa

La resta inversa utiliza el mismo principio lógico que la resta convencional, pero intercambiando el orden de los operandos.

La operación requerida es:

$$B - A$$

Aplicando complemento a dos:

$$R = B + \overline{A} + 1$$

Por lo tanto, no es necesario diseñar un nuevo circuito aritmético. Se reutiliza la misma estructura de inversión y suma, modificando el orden de las entradas.

5.4. Multiplexor para selección del segundo operando

Antes de ingresar a la ALU, el segundo operando puede provenir del valor ingresado por el usuario o del resultado almacenado anteriormente.

Para cada bit se utiliza un multiplexor 2:1.

Sean las entradas X_0 y X_1 , y la señal de selección S .

La tabla de verdad es:

Cuadro 6: Tabla de verdad de un multiplexor 2:1

S	Salida Y
0	X_0
1	X_1

La expresión booleana correspondiente es:

$$Y = \overline{S}X_0 + SX_1$$

Por lo tanto, cada multiplexor puede implementarse utilizando:

- una compuerta NOT;
- dos compuertas AND;
- una compuerta OR.

Para seleccionar un operando de cuatro bits se utilizan cuatro multiplexores de un bit en paralelo.

Cuando la señal de selección del resultado previo se encuentra en cero:

$$B_{sel} = B_{ext}$$

Cuando se encuentra en uno:

$$B_{sel} = R_{prev}$$

5.5. Desplazamiento lógico a la izquierda

El desplazamiento lógico a la izquierda se implementa sin utilizar directamente el operador de desplazamiento de Verilog.

El circuito utiliza los dos bits menos significativos del segundo operando:

$$s_1s_0 = B[1 : 0]$$

El desplazamiento se construye mediante dos etapas.

La primera etapa permite elegir entre:

- desplazamiento de cero posiciones;
- desplazamiento de una posición.

La segunda etapa permite agregar:

- cero posiciones;
- dos posiciones adicionales.

De esta forma, se obtienen las cuatro posibilidades:

Cuadro 7: Desplazamiento lógico a la izquierda

s_1	s_0	Resultado
0	0	A
0	1	desplazamiento de 1 posición
1	0	desplazamiento de 2 posiciones
1	1	desplazamiento de 3 posiciones

En un desplazamiento lógico a la izquierda se introducen ceros por el bit menos significativo.

Por ejemplo:

$$A = 0011$$

con:

$$B[1 : 0] = 01$$

produce:

$$R = 0110$$

Para dos posiciones:

$$0011 \rightarrow 1100$$

y para tres posiciones:

$$0011 \rightarrow 1000$$

Cada etapa se implementa mediante combinaciones de compuertas NOT, AND y OR que actúan como multiplexores.

5.6. Desplazamiento lógico a la derecha

El desplazamiento lógico a la derecha utiliza una arquitectura análoga.

La primera etapa selecciona entre cero y una posición de desplazamiento, mientras que la segunda permite agregar cero o dos posiciones.

Cuadro 8: Desplazamiento lógico a la derecha

s_1	s_0	Resultado
0	0	A
0	1	desplazamiento de 1 posición
1	0	desplazamiento de 2 posiciones
1	1	desplazamiento de 3 posiciones

Al tratarse de un desplazamiento lógico, se introducen ceros desde el bit más significativo.

Por ejemplo:

$$A = 1100$$

con:

$$B[1 : 0] = 01$$

produce:

$$R = 0110$$

Para dos posiciones:

$$1100 \rightarrow 0011$$

y para tres posiciones:

$$1100 \rightarrow 0001$$

5.7. Selección de operaciones en la ALU

Todas las operaciones descritas anteriormente se calculan mediante bloques independientes.

La ALU utiliza el código de operación:

$$O_2O_1O_0$$

para activar solamente la ruta correspondiente.

Las funciones de selección pueden expresarse como:

$$SEL_{SUM} = \overline{O_2}\overline{O_1}O_0$$

$$SEL_{SUB} = \overline{O_2}O_1\overline{O_0}$$

$$SEL_{RSUB} = \overline{O_2}O_1O_0$$

$$SEL_{SHL} = O_2\overline{O_1}\overline{O_0}$$

$$SEL_{SHR} = O_2\overline{O_1}O_0$$

Para cada bit i del resultado se generan las rutas:

$$P_{SUM,i} = SUM_i \cdot SEL_{SUM}$$

$$P_{SUB,i} = SUB_i \cdot SEL_{SUB}$$

$$P_{RSUB,i} = RSUB_i \cdot SEL_{RSUB}$$

$$P_{SHL,i} = SHL_i \cdot SEL_{SHL}$$

$$P_{SHR,i} = SHR_i \cdot SEL_{SHR}$$

La salida de la ALU se obtiene mediante una compuerta OR:

$$R_i = P_{SUM,i} + P_{SUB,i} + P_{RSUB,i} + P_{SHL,i} + P_{SHR,i}$$

Esta misma estructura se replica para los cuatro bits.

Para el código:

000

ninguna de las cinco rutas anteriores se encuentra activa.

Por lo tanto:

$$R = 0000$$

lo que implementa la operación de reinicio del resultado.

6. Almacenamiento del resultado y realimentación

La salida combinacional de la ALU no se utiliza directamente como resultado permanente.

El sistema incorpora un registro de cuatro bits encargado de conservar el último resultado ejecutado.

Cuando se genera la señal de ejecución, el valor presente en la salida de la ALU se almacena en el registro.

Por lo tanto:

$$R_{almacenado} \leftarrow R_{ALU}$$

El registro cumple un papel fundamental para permitir operaciones consecutivas.

Su salida se conecta simultáneamente:

- a la interfaz de visualización;
- al multiplexor de selección del segundo operando.

De esta forma, si el usuario selecciona la opción de resultado anterior:

$$B = R_{almacenado}$$

Esto permite, por ejemplo, ejecutar primero:

$$3 + 2 = 5$$

y utilizar posteriormente ese resultado para calcular:

$$1 + R_{prev} = 1 + 5 = 6$$

sin volver a ingresar manualmente el valor 5.

7. Interfaz con la FPGA

La implementación física utiliza los cuatro botones, cuatro LEDs y dos displays de siete segmentos disponibles en la Nandland Go Board.

7.1. Botones de entrada

Cuadro 9: Asignación funcional de botones

Botón	Función
SW1	Incrementar el valor actualmente seleccionado
SW2	Disminuir el valor actualmente seleccionado
SW3	Confirmar el valor y avanzar al siguiente estado
SW4	Seleccionar el resultado de la operación anterior como segundo operando

El valor editable utiliza cuatro bits, por lo que su incremento y decremento presentan comportamiento módulo 16.

Por ejemplo:

$$0000 - 1 = 1111$$

El patrón:

$$1111$$

representa -1 en complemento a dos.

7.2. Tratamiento de los botones

Los botones físicos pueden generar múltiples transiciones eléctricas durante una sola pulsación debido al rebote mecánico.

Para evitar que una pulsación sea interpretada varias veces, el sistema incorpora un bloque de *debounce*.

Una vez estabilizada la señal, un segundo bloque genera un único pulso por pulsación.

Este mecanismo permite que cada acción sobre SW1, SW2, SW3 o SW4 corresponda a un solo evento dentro del sistema digital.

7.3. Indicadores LED

Los primeros tres LEDs muestran el código de operación seleccionado.

Cuadro 10: Uso de LEDs

Indicador	Función
D1–D3	Código binario de la operación seleccionada
D4	Indica que el resultado anterior está seleccionado como segundo operando

7.4. Displays de siete segmentos

Los dos displays se utilizan para representar el valor que está siendo ingresado o el resultado de la operación.

El primer display indica el signo negativo cuando corresponde.

El segundo display representa la magnitud del valor utilizando formato hexadecimal.

Por ejemplo, para el patrón de cuatro bits:

1110

la interpretación en complemento a dos es:

−2

Por lo tanto, la representación visual utiliza:

- signo negativo en el primer display;

- dígito 2 en el segundo display.

8. Restricciones de implementación

Uno de los objetivos principales del proyecto es desarrollar la lógica combinacional a nivel de compuertas.

Por esta razón, los módulos sintetizables fueron contruidos evitando operadores y estructuras de alto nivel para las operaciones solicitadas.

Las operaciones combinacionales principales utilizan primitivas como:

- `and`;
- `or`;
- `not`;
- `xor`;
- `buf`.

La suma se construye mediante `full_adder` conectados en cascada.

La resta se obtiene mediante complemento a dos.

Los desplazamientos se implementan mediante redes de selección contruidas con compuertas.

La selección de operaciones de la ALU se realiza mediante decodificación lógica del código de operación.

Los operadores de alto nivel pueden utilizarse dentro de los *testbenches*, debido a que estos archivos se utilizan únicamente para verificación y no forman parte del hardware sintetizado.

9. Conclusiones

El desarrollo del proyecto permitió implementar una calculadora digital completa utilizando una arquitectura modular de cuatro bits sobre una FPGA.

La construcción del sistema mediante compuertas lógicas permitió analizar de manera directa el funcionamiento interno de operaciones que normalmente son abstraídas por los operadores de alto nivel de los lenguajes de descripción de hardware.

La implementación del `full_adder` permitió construir un sumador de cuatro bits mediante propagación de acarreo. A partir de este mismo bloque fue posible implementar las operaciones de resta y resta inversa utilizando complemento a dos.

Los desplazamientos lógicos fueron implementados mediante una estructura de dos etapas, permitiendo seleccionar desplazamientos entre cero y tres posiciones sin utilizar directamente operadores de desplazamiento.

La ALU integra todos estos circuitos mediante una red de decodificación del código de operación y compuertas lógicas de selección.

Además, el uso de un registro independiente para almacenar el resultado permite reutilizar el valor de una operación anterior como segundo operando, entregando al sistema una funcionalidad similar a una calculadora secuencial básica.

Finalmente, la integración de una máquina de estados, tratamiento de botones, LEDs y displays permite conectar la lógica digital desarrollada con los recursos físicos disponibles en la Nandland Go Board.

10. Declaración de uso de Inteligencia Artificial

Declaración de Uso de Herramientas de Inteligencia Artificial

10.1. Identificación de la actividad

- **Curso:** Arquitectura de Computadores.
- **Código del curso:** 433.
- **Nombre de la actividad:** Proyecto 1 — Diseño Lógico y FPGA.
- **Fecha de entrega:** 8/09/2026.
- **Estudiantes:** Cristian Acevedo, Guillermo Arriaza y Eduardo Ponce.

10.2. Declaración sobre uso de IA

☐ Sí se utilizaron herramientas de inteligencia artificial según se detalla a continuación.

10.3. Descripción de uso de IA

Herramienta empleada	Inteligencia Artificial
Nombre de la IA	ChatGPT
Fechas de uso	Agosto y septiembre de 2026
Propósito del uso	Apoyo en redacción, revisión ortográfica, organización del informe y análisis de problemas asociados a la implementación.
Descripción de integración	La herramienta se utilizó como apoyo para revisar textos, organizar explicaciones técnicas y analizar dificultades durante el desarrollo. El código, las decisiones de diseño, las pruebas y el contenido final fueron revisados por los integrantes del grupo.

10.4. Declaración de responsabilidad académica

Los firmantes de este documento declaramos que:

- Hemos revisado críticamente todo el contenido entregado.
- Comprendemos plenamente el trabajo presentado y somos capaces de explicarlo y defenderlo académicamente.
- Asumimos responsabilidad por la calidad, veracidad y originalidad del contenido entregado, independientemente del uso o no uso de herramientas de inteligencia artificial.
- La información declarada en este documento es veraz.

Cristian Acevedo Guillermo Arriaza Eduardo Ponce